



CH9 — Pointers and Strings



SLIDE 1 — What Are Pointers and Strings?

A **string** is a collection of characters stored in an array.

Example:

None

```
char str[] = "HELLO";
```

A pointer can point to the first character of the string.

None

```
char *p = str;
```

Now:

None

```
str → H E L L O \0
      ↑
      p
```



Easy Analogy:

Think of a string as a row of boxes containing letters.

The pointer tells us **where the first box is**.



Remember:

String → Characters

Pointer → Address

***p** → Character



SLIDE 2 — Accessing String Characters

Suppose:

None

```
char str[] = "HELLO";  
char *p = str;
```

We can access characters using the pointer.

None

`*p`

gives:

None

`H`

Move the pointer:

None

`p++;`

Now:

None

`*p`

gives:

None

`E`

 **Look at this:**

None

`H   E   L   L   O   \0`

`↑`

p

p++ ↓

H E L L O \0

↑
p

💡 **Memory Trick:**

p++ → **MOVE FORWARD**

*p → **GET CHARACTER**

SLIDE 3 — Using ***(p+i)** With Strings

We can access different characters using pointer arithmetic.

None

```
char str[] = "HELLO";  
char *p = str;
```

Then:

None

```
*(p + 0) → H  
*(p + 1) → E  
*(p + 2) → L  
*(p + 3) → L  
*(p + 4) → O
```

Easy Analogy

Think of the string as numbered boxes:

None

Index:	0	1	2	3	4
	↓	↓	↓	↓	↓
String:	H	E	L	L	O

`p + 2` moves to position 2.

`*(p + 2)` gets the character there.

★ **Remember:**

`*(p+i)` → CHARACTER AT POSITION `i`

SLIDE 4 — Traversing a String Using a Pointer

We can use a pointer to go through the string character by character.

None

```
#include <stdio.h>

int main()
{
    char str[] = "HELLO";
    char *p = str;

    while (*p != '\0')
    {
        printf("%c", *p);
        p++;
    }

    return 0;
}
```

Output:

None

HELLO

What's happening?

None

```
*p → H → print  
p++ → move
```

```
*p → E → print  
p++ → move
```

```
*p → L → print  
p++ → move
```

...

The loop stops when it reaches:

None

'\0'

Memory Trick:

CHECK → PRINT → MOVE → REPEAT

SLIDE 5 — RECAP

Pointers and Strings

Concept	Meaning
<code>char str[]</code>	Creates a string
<code>char *p = str</code>	Pointer points to first character

<code>*p</code>	Gets current character
<code>p++</code>	Moves to next character
<code>*(p+i)</code>	Gets character at position <code>i</code>
<code>'\0'</code>	End of string

BIG MEMORY TRICK

STRING → POINTER → GET → MOVE → REPEAT

None

`"HELLO"`

`H → E → L → L → O → \0`

↑

`p`

★ TAKEAWAY:

A pointer can move through a string **one character at a time**.



QUESTIONS

1 What does `char *p = str` do?

- A) Deletes the string
- B) Makes `p` point to the first character of the string
- C) Moves `p` to the last character
- D) Changes the string to a number



Hint: `str` represents the starting location of the string.



Glossary:

Character → A single letter, number, or symbol.

2 What is the output?

None

```
char str[] = "HELLO";
```

```
char *p = str;
```

```
printf("%c", *p);
```

A) H

B) E

C) L

D) O

 **Hint:** `p` initially points to the first character.

 **Glossary:**

String → A sequence of characters stored together.

3 Which statement accesses the fourth character?

A) `*p`

B) `*(p + 1)`

C) `*(p + 2)`

D) `*(p + 3)`

 **Hint:** The first character is at position `0`.

 **Glossary:**

Index → The position number of an element.

4 Complete the Code

Print every character of the string using the pointer.

None

```
char str[] = "HELLO";

char *p = str;

while (*p != '\0')
{
    printf("%c", ____);
    p++;
}
```

 **Hint:** Print the character currently pointed to by `p`.

 **Glossary:**

`'\0'` → The special character that marks the end of a string.

5 What is the output?

None

```
char str[] = "CODE";

char *p = str;

for (int i = 0; i < 4; i++)
{
    printf("%c", *(p + i));
}
```

- A) CODE
- B) EDOC
- C) CDOE
- D) CO

 **Hint:** `*(p+i)` accesses each character in order.

 **Glossary:**

Traversal → Visiting each character one by one.